

Memoria de la práctica

Analizador distribuido de vídeo con MPI y GPU

Asignatura: Arquitecturas Paralelas

May 17, 2026

Abstract

Este documento describe la arquitectura, funcionamiento e implementación del proyecto *MPI Video Analyzer*, una aplicación C++ para analizar vídeos y detectar personas por frame mediante modelos YOLO11 exportados a ONNX. El interés principal de la práctica está en el uso combinado de paralelismo distribuido con MPI y aceleración de cómputo en GPU mediante CUDA u OpenCL. La aplicación divide el vídeo en lotes de frames, reparte dichos lotes entre procesos worker, ejecuta el preprocesado y la inferencia en paralelo, y agrega los resultados en el proceso maestro para producir ficheros CSV, JSON y opcionalmente un vídeo anotado.

Contents

1	Introducción	3
2	Resumen del proyecto	3
3	Tecnologías utilizadas	3
4	Organización del código	4
5	Arquitectura paralela	4
5.1	Modelo SPMD	4
5.2	Descomposición del problema	5
5.3	Topología maestro-worker	5
5.4	Planificadores	5
6	Flujo de ejecución	6
6.1	Arranque	6
6.2	Trabajo del maestro	6
6.3	Trabajo de cada worker	6
7	Backends de GPU	7
7.1	Ruta CUDA	7
7.2	Ruta OpenCL	8
7.3	Backend automático	8
8	Modelo de detección YOLO11	8
9	Estructuras de datos	8
10	Métricas y salidas	8
11	Compilación	9
12	Ejecución	9
13	Benchmarking	10
14	Análisis desde Arquitecturas Paralelas	11
14.1	Granularidad	11
14.2	Comunicación frente a cómputo	11
14.3	Balanceo de carga	11
14.4	Heterogeneidad	12
14.5	Cuellos de botella	12
15	Limitaciones	12
16	Posibles mejoras	13
17	Conclusiones	13
A	Parámetros principales	13

1 Introducción

El objetivo del proyecto es procesar un vídeo de entrada para detectar y contar personas en cada frame. La aplicación está pensada para escenarios de vigilancia o rescate, donde el tiempo de procesamiento puede reducirse distribuyendo el trabajo entre varios procesos y, cuando existen dispositivos disponibles, acelerando las fases más costosas en GPU.

Desde el punto de vista de Arquitecturas Paralelas, el programa combina dos niveles de paralelismo:

- **Paralelismo de grano grueso con MPI:** el vídeo se divide en lotes de frames y cada proceso worker procesa lotes independientes.
- **Paralelismo masivo en GPU:** cada worker puede usar CUDA u OpenCL para acelerar el preprocesado de imagen y la inferencia del modelo neuronal.

El resultado no es solo una aplicación funcional de visión por computador, sino también una plataforma para estudiar reparto de carga, escalabilidad, sobrecostes de comunicación, tamaño de lote y heterogeneidad entre dispositivos.

2 Resumen del proyecto

El ejecutable principal se llama `rescue_video_analyzer`. Se invoca con un vídeo de entrada, un modelo YOLO11 en formato ONNX y varios parámetros de ejecución. Su salida principal se escribe en el directorio configurado mediante `-output-dir`.

- `frame_results.csv`: métricas por frame, incluyendo rank que lo procesó, GPU usada, conteo de personas y tiempos por etapa.
- `summary.json`: resumen global con metadatos del vídeo, configuración MPI, estadísticas de detección, balanceo de carga y tiempos totales.
- `annotated_people.mp4` o `annotated_people.avi`: vídeo anotado con cajas de detección y conteo visual, salvo que se use `-no-annotated-video`.

3 Tecnologías utilizadas

La práctica integra varias tecnologías de computación paralela y visión artificial.

Tecnología	Uso en el proyecto
C++17	Lenguaje principal de la aplicación.
OpenMPI	Ejecución distribuida mediante procesos MPI.
CUDA	Backend principal para preprocesado GPU e inferencia con ONNX Runtime CUDA.
OpenCL	Backend alternativo mediante OpenCV UMat y OpenCV DNN.
OpenCV	Lectura/escritura de vídeo, transformaciones de imagen, NMS y generación del vídeo anotado.
ONNX Runtime	Ejecución del modelo YOLO11 en el backend CUDA.

YOLO11 ONNX	Modelo de detección de objetos; se filtra la clase persona.
CMake	Sistema de compilación del proyecto.

4 Organización del código

La estructura principal del repositorio es la siguiente:

```
.
|-- CMakeLists.txt
|-- README.md
|-- dataset/ # videos de prueba
|-- include/rescue/ # cabeceras publicas
|-- models/ # modelos YOLO11 en ONNX
|-- scripts/
| |-- benchmark_parallel.sh
| '-- launch_cluster.py
'-- src/ # implementacion C++, CUDA y OpenCL
```

Los módulos más importantes son:

- `src/main.cpp`: parseo de argumentos, preparación del entorno y arranque MPI.
- `src/mpi_runtime.cpp`: lógica maestro-worker, planificadores y comunicación MPI.
- `src/gpu_preprocess.cu`: kernels CUDA de preprocesado.
- `src/gpu_preprocess_opencv.cpp`: preprocesado alternativo con OpenCV/OpenCL.
- `src/yolo_detector_cuda.cpp`: inferencia YOLO11 con ONNX Runtime CUDA.
- `src/yolo_detector_opencv.cpp`: inferencia YOLO11 con OpenCV DNN y OpenCL.
- `src/output_writer.cpp`: generación de CSV, JSON y vídeo anotado.
- `src/tracker.cpp`: tracker sencillo usado solo para visualización.

5 Arquitectura paralela

5.1 Modelo SPMD

La aplicación sigue un modelo SPMD (*Single Program, Multiple Data*): todos los procesos ejecutan el mismo binario, pero su comportamiento depende del rank MPI.

- El **rank 0** actúa como proceso maestro.
- Los **ranks 1..N** actúan como workers.

El maestro no ejecuta inferencia. Su trabajo principal es leer frames, construir lotes, enviarlos a los workers, recibir resultados y escribir las salidas. Los workers reciben lotes, ejecutan el preprocesado y la detección, y devuelven un paquete de resultados por frame.

5.2 Descomposición del problema

El problema se descompone por frames. Cada frame puede procesarse de forma independiente para obtener detecciones de personas, por lo que existe paralelismo de datos natural. En vez de enviar frames individuales, el programa agrupa frames en lotes de tamaño `batch_size`. Esto reduce el número de mensajes MPI y permite amortizar parte del coste de comunicación.

Si el vídeo tiene F frames y el tamaño de lote es B , el número aproximado de tareas es:

$$T = \left\lceil \frac{F}{B} \right\rceil$$

Cada tarea contiene un `BatchHeader` y un bloque de bytes con los frames codificados como matrices `CV_8UC3`.

5.3 Topología maestro-worker

La comunicación MPI utiliza etiquetas separadas para cabeceras y cargas:

Etiqueta	Sentido	Contenido
<code>kTagBatchHeader</code>	Maestro $\rightarrow$ worker	Cabecera del lote.
<code>kTagBatchPayload</code>	Maestro $\rightarrow$ worker	Bytes de los frames.
<code>kTagResultHeader</code>	Worker $\rightarrow$ maestro	Cabecera del resultado.
<code>kTagResultPayload</code>	Worker $\rightarrow$ maestro	Array de <code>FramePacket</code> .

Table 2: Mensajes MPI principales.

Las estructuras simples se envían como tipos MPI contiguos de bytes, ya que `BatchHeader`, `ResultHeader`, `FrameResult` y `FramePacket` son trivialmente copiables. Esta decisión simplifica el transporte de datos estructurados, aunque presupone que los procesos ejecutan una versión binaria compatible de la aplicación.

5.4 Planificadores

El proyecto implementa tres modos de scheduling:

- **dynamic**: el maestro entrega un nuevo lote al worker que acaba de terminar. Es el modo recomendado en entornos heterogéneos porque se adapta al ritmo real de cada GPU.
- **static-contiguous**: cada worker recibe un bloque contiguo de frames. Reduce decisiones en tiempo de ejecución, pero puede producir desequilibrio si unos frames son más costosos que otros o si los dispositivos no tienen el mismo rendimiento.
- **static-round-robin**: los lotes se asignan por turnos a los workers. Reparte mejor que el bloque contiguo cuando el coste cambia a lo largo del vídeo, aunque no reacciona al rendimiento real de cada worker.

El scheduling dinámico es especialmente relevante en Arquitecturas Paralelas porque reduce el tiempo ocioso de los workers. Cuando un worker termina, el maestro recibe su resultado con `MPI_Probe(MPI_ANY_SOURCE, ...)` y le asigna trabajo nuevo si quedan lotes pendientes.

6 Flujo de ejecución

6.1 Arranque

El programa valida los argumentos de línea de comandos y comprueba que existan tanto el vídeo de entrada como el modelo ONNX. También prepara rutas de bibliotecas dinámicas para ONNX Runtime y CUDA, especialmente en entornos WSL o virtualenv.

Si el binario se ejecuta sin `mpirun`, puede relanzarse automáticamente con un mundo MPI por defecto. En CUDA, el esquema natural es:

$$\text{world_size} = 1 + \text{numero_de_GPUs}$$

donde el proceso adicional es el maestro.

6.2 Trabajo del maestro

El maestro realiza los siguientes pasos:

1. Abre el vídeo con OpenCV y obtiene metadatos: resolución, FPS, número de frames y duración.
2. Construye un plan de lotes según `batch-size`.
3. Envía un primer lote a cada worker disponible.
4. Espera resultados de cualquier worker.
5. Inserta los `FramePacket` recibidos en un vector global.
6. Envía un nuevo lote al worker que ha quedado libre, o una señal de parada si ya no hay trabajo.
7. Ordena los resultados por `frame_index`.
8. Escribe las salidas CSV, JSON y vídeo anotado.

Este diseño centraliza la entrada/salida de vídeo en el maestro y evita que varios procesos lean simultáneamente el mismo fichero. A cambio, el maestro se convierte en punto de coordinación y en posible cuello de botella si el número de workers crece mucho.

6.3 Trabajo de cada worker

Cada worker ejecuta un bucle hasta recibir una cabecera de lote con `frame_count = 0`. Para cada lote:

1. Recibe la cabecera y el payload de frames.

2. Materializa vistas `cv::Mat` sobre el bloque de bytes recibido.
3. Realiza una fase de calentamiento del backend si todavía no se ha hecho.
4. Redimensiona cada frame a la resolución de procesamiento si es necesario.
5. Ejecuta el preprocesado GPU: *letterbox*, normalización, conversión a tensor y métricas auxiliares.
6. Ejecuta la inferencia YOLO11.
7. Filtra detecciones de clase persona y aplica NMS cuando corresponde.
8. Construye un `FramePacket` por frame y lo devuelve al maestro.

7 Backends de GPU

7.1 Ruta CUDA

La ruta CUDA es la ruta principal del proyecto. Se compila si `RESCUE_ENABLE_CUDA=ON` en CMake. En esta ruta, cada worker activa una GPU mediante `cudaSetDevice`. Si no se fuerza un dispositivo con `-gpu-device`, el programa usa el rank local del worker para repartir workers entre GPUs visibles:

$$\text{device_id} = \text{local_worker_rank} \bmod \text{visible_device_count}$$

El preprocesado CUDA se implementa en `src/gpu_preprocess.cu`. El kernel principal:

- Lee la imagen BGR de entrada.
- Calcula el reescalado con *letterbox*, manteniendo la relación de aspecto.
- Aplica interpolación bilineal.
- Convierte BGR a RGB.
- Normaliza los valores a $[0, 1]$.
- Escribe el tensor en formato `NCHW`, esperado por YOLO.
- Genera una imagen en gris auxiliar para calcular intensidad media y densidad de bordes.

Un segundo kernel calcula estadísticas con operaciones atómicas: suma de intensidades y conteo de píxeles cuyo gradiente supera el umbral `edge-threshold`. La inferencia se realiza con ONNX Runtime y el *CUDA Execution Provider*. El tensor de entrada se crea directamente sobre memoria CUDA, evitando copiarlo a CPU antes de ejecutar el modelo.

7.2 Ruta OpenCL

La ruta OpenCL es una alternativa basada en OpenCV. Usa `cv::UMat` para permitir que OpenCV ejecute operaciones sobre OpenCL cuando está disponible. El pre-procesado incluye redimensionado, *letterbox*, creación del blob y cálculo de métricas de imagen con Sobel. La inferencia se realiza con `cv::dnn::readNetFromONNX` y target `DNN_TARGET_OPENCL`. Si OpenCL no está disponible, OpenCV puede caer a CPU.

7.3 Backend automático

Con `-gpu-backend auto`, el worker prefiere CUDA si hay dispositivos visibles y el proyecto fue compilado con soporte CUDA. En caso contrario, utiliza la ruta OpenCL/CPU. Este comportamiento facilita ejecutar el mismo binario en equipos con distintas capacidades.

8 Modelo de detección YOLO11

Los modelos incluidos en `models/` están exportados a ONNX. El modelo por defecto es:

```
models/yolo11s_1088.onnx
```

El parámetro `-resize-width` debe coincidir con el tamaño de entrada usado al exportar el modelo. Por ejemplo, `yolo11s_1088.onnx` se utiliza con `-resize-width 1088`. Tras la inferencia, el programa decodifica la salida de YOLO, filtra la clase persona (`class_id = 0`) y descarta detecciones con confianza inferior a `-score-threshold`. En salidas no end-to-end se aplica *Non-Maximum Suppression* con `-nms-threshold` y `-top-k`.

9 Estructuras de datos

Estructura	Función
<code>AppConfig</code>	Configuración global: rutas, tamaño de lote, backend GPU, modelo, thresholds y scheduler.
<code>VideoMetadata</code>	Metadatos del vídeo: resolución, FPS, frames y duración.
<code>BatchHeader</code>	Cabecera enviada por MPI para describir un lote de frames.
<code>FrameResult</code>	Resultado numérico de un frame: rank, GPU, conteo y tiempos.
<code>DetectionBox</code>	Caja de detección con coordenadas, confianza y clase.
<code>FramePacket</code>	Paquete completo devuelto al maestro: <code>FrameResult</code> y hasta 128 detecciones.

El límite `kMaxDetectionsPerFrame = 128` acota el tamaño del paquete MPI por frame. Esto simplifica la comunicación porque todos los `FramePacket` tienen tamaño fijo.

10 Métricas y salidas

La aplicación registra tiempos por frame:

- `read_ms`: coste medio asociado a recibir el lote desde MPI.
- `preprocess_ms`: tiempo de redimensionado/preprocesado.
- `detection_ms`: tiempo de inferencia y decodificación.
- `processing_ms`: suma de recepción, preprocesado e inferencia por frame.

En `summary.json` también se escriben métricas globales:

- `distributed_processing_ms`: tiempo desde que el maestro empieza a repartir trabajo hasta que recibe todos los resultados.
- `output_write_ms`: tiempo de escritura de salidas.
- `total_wall_ms`: suma de procesamiento distribuido y escritura.
- `load_balance`: frames procesados por worker, mínimo, máximo y ratio máximo/-media.
- `worker_stats`: estadísticas desglosadas por rank.

Estas métricas permiten analizar si el programa escala, si hay workers ociosos, si el cuello de botella está en la inferencia, en el preprocesado, en la comunicación MPI o en la escritura de resultados.

11 Compilación

La compilación normal usa CMake:

```
cmake -S . -B build
cmake --build build -j
```

Para compilar sin CUDA y usar solo OpenCL/CPU:

```
cmake -S . -B build -DRESCUE_ENABLE_CUDA=OFF
cmake --build build -j
```

La arquitectura CUDA por defecto en el proyecto es 89, adecuada para la GPU usada durante el desarrollo. Puede cambiarse con:

```
cmake -S . -B build -DCMAKE_CUDA_ARCHITECTURES=86
cmake --build build -j
```

12 Ejecución

La ejecución recomendada en una máquina con una GPU NVIDIA es:

```
mpirun -np 2 ./build/rescue_video_analyzer \
--input dataset/videoset2.mp4 \
--output-dir output \
--batch-size 8 \
--scheduler dynamic
```

Para medir rendimiento sin incluir el coste de generar vídeo anotado:

```
mpirun -np 2 ./build/rescue_video_analyzer \  
  --input dataset/videoset2.mp4 \  
  --output-dir output_bench \  
  --batch-size 8 \  
  --scheduler dynamic \  
  --no-annotated-video
```

El modo cluster puede lanzarse con el script:

```
./scripts/launch_cluster.py \  
  --ips localhost,192.168.1.55 \  
  --args "--input dataset/videoset2.mp4 --output-dir output_cluster --batch-size  
  8 --scheduler dynamic"
```

Este script sondea GPUs NVIDIA y OpenCL en los nodos, genera un `hosts_cluster.txt` y lanza `mpiexec`. En un cluster real, las rutas de `dataset/`, `models/` y del ejecutable deben estar disponibles en todos los nodos o en un sistema de ficheros compartido.

13 Benchmarking

El script `scripts/benchmark_parallel.sh` automatiza pruebas variando el tamaño del mundo MPI y el scheduler:

```
MPI_WORLD_SIZES="2 3 4" \  
SCHEDULERS="dynamic static-contiguous static-round-robin" \  
./scripts/benchmark_parallel.sh dataset/videoset.mp4 parallel_benchmarks
```

El script escribe una tabla TSV con:

- `world_size`
- `scheduler`
- `total_wall_ms`
- `distributed_processing_ms`
- `average_detection_ms`
- `average_processing_ms`
- `average_people_per_frame`
- directorio de salida

Una prueba local recogida en el README, usando `dataset/videoset2.mp4`, `mpirun -np 2, -batch-size 8 y -no-annotated-video`, obtuvo:

Métrica	Valor
Personas medias por frame	29.863
Preprocesado medio	3.841 ms
Detección media	16.158 ms
Procesamiento medio	20.090 ms
Tiempo total	6399.037 ms

Table 4: Ejemplo de resultado local documentado.

Es importante interpretar estos datos con cuidado. En una máquina con una sola GPU, ejecutar `np=3` o `np=4` no implica escalado multi-GPU real: varios workers competirán por el mismo dispositivo o caerán a rutas alternativas. Para medir la ruta CUDA principal en una sola GPU, la configuración más limpia es `np=2`: un maestro y un worker.

14 Análisis desde Arquitecturas Paralelas

14.1 Granularidad

La unidad de trabajo es el lote de frames. Un lote demasiado pequeño incrementa el número de mensajes MPI y el coste de scheduling. Un lote demasiado grande reduce la capacidad del scheduler dinámico para balancear carga, porque un worker lento puede quedarse con una tarea grande durante demasiado tiempo. El parámetro `-batch-size` permite estudiar ese compromiso.

14.2 Comunicación frente a cómputo

Cada lote requiere enviar al worker los bytes completos de los frames. Para un frame CV_8UC3, el tamaño aproximado es:

$$S = ancho \cdot alto \cdot 3 \text{ bytes}$$

Por tanto, el coste de comunicación crece con la resolución de procesamiento y el tamaño del lote. Sin embargo, el coste dominante suele estar en inferencia YOLO, especialmente con modelos grandes o resoluciones de entrada altas. El diseño busca que la comunicación quede amortizada por el coste computacional de cada lote.

14.3 Balanceo de carga

El modo dinámico mejora el balanceo cuando:

- Hay GPUs con distinto rendimiento.
- Algún nodo cae a OpenCL/CPU.
- El coste de detección varía por frame.
- Hay contención por compartir dispositivo.

El fichero `summary.json` incluye el ratio:

$$\text{max_to_mean_ratio} = \frac{\text{frames del worker con más carga}}{\text{media de frames por worker activo}}$$

Un valor cercano a 1 indica buen balanceo en número de frames. Aun así, el número de frames no siempre equivale a tiempo: dos workers pueden procesar el mismo número de frames con tiempos distintos si sus GPUs no son equivalentes.

14.4 Heterogeneidad

El proyecto contempla heterogeneidad de hardware. El backend automático decide entre CUDA y OpenCL/CPU según disponibilidad. Además, el script de cluster asigna un slot por GPU detectada y un slot adicional para el maestro en el nodo inicial. Esta aproximación permite usar varios nodos con distinta configuración, aunque la eficiencia final depende de que el scheduler mantenga ocupados los dispositivos rápidos sin quedar bloqueado por los lentos.

14.5 Cuellos de botella

Los posibles cuellos de botella son:

- **Maestro:** lee el vídeo, serializa lotes y recibe resultados. Con muchos workers puede limitar la escalabilidad.
- **Comunicación MPI:** el envío de frames completos puede ser costoso en red.
- **GPU:** la inferencia YOLO es la fase más pesada por frame.
- **Memoria:** hay copias CPU-GPU en preprocesado CUDA y almacenamiento temporal de frames.
- **Salida:** la generación de vídeo anotado puede distorsionar benchmarks, por eso existe `-no-annotated-video`.

15 Limitaciones

- El maestro centraliza la lectura del vídeo, lo que simplifica la coherencia pero puede limitar escalabilidad en clusters grandes.
- Los frames se envían completos por MPI; no hay compresión ni lectura distribuida desde cada nodo.
- `FramePacket` fija un máximo de 128 detecciones por frame para facilitar la comunicación.
- El tracking no se usa para modificar el cómputo distribuido: solo para el vídeo anotado y el acumulado visual.
- La ruta OpenCL depende de la disponibilidad real de OpenCL en OpenCV; si no existe, puede ejecutarse en CPU.

16 Posibles mejoras

- Usar comunicación no bloqueante con `MPI_Isend` y `MPI_Irecv` para solapar comunicación y cómputo.
- Permitir lectura distribuida del vídeo si el formato y el almacenamiento compartido lo permiten.
- Enviar frames ya redimensionados para reducir bytes transmitidos, comparando el coste de mover preprocesado al maestro frente a mantenerlo en workers.
- Agrupar inferencias por batch real en GPU para aprovechar mejor el throughput del modelo.
- Implementar afinidad más fina entre ranks, NUMA y GPUs en nodos multi-GPU.
- Registrar trazas temporales completas para representar gráficamente ocupación de workers y fases de pipeline.

17 Conclusiones

El proyecto implementa una arquitectura paralela híbrida adecuada para una práctica de Arquitecturas Paralelas. A nivel distribuido, usa MPI para repartir lotes de frames entre workers independientes. A nivel de dispositivo, emplea CUDA u OpenCL para acelerar el preprocesado y la inferencia de un modelo YOLO11. El diseño permite comparar schedulers, medir balanceo de carga y observar la relación entre comunicación, cómputo y escritura de resultados.

La principal fortaleza del sistema es que el problema tiene paralelismo de datos natural: los frames pueden procesarse de forma independiente y agregarse posteriormente ordenando por `frame_index`. El scheduler dinámico aprovecha esta independencia para adaptarse a hardware heterogéneo. Como limitación, el maestro concentra la lectura y distribución de frames, por lo que una mejora futura clara sería estudiar técnicas de solapamiento, comunicación no bloqueante y lectura distribuida.

A Parámetros principales

Parámetro	Descripción
<code>-input <video></code>	Vídeo de entrada. Obligatorio.
<code>-output-dir <dir></code>	Directorio de salida.
<code>-batch-size <N></code>	Frames por lote MPI.
<code>-processing-width <px></code>	Ancho de procesamiento; 0 usa resolución nativa.
<code>-processing-height <px></code>	Alto de procesamiento; 0 usa resolución nativa.
<code>-gpu-device auto <id></code>	Selección de GPU visible.
<code>-gpu-backend auto cuda opencl</code>	Backend de cómputo usado por los workers.
<code>-resize-width <px></code>	Tamaño cuadrado de entrada YOLO.
<code>-edge-threshold <N></code>	Umbral para densidad de bordes auxiliar.

<code>-model <path></code>	Modelo ONNX.
<code>-score-threshold <0..1></code>	Confianza mínima de detección.
<code>-nms-threshold <0..1></code>	Umbral de NMS.
<code>-top-k <N></code>	Máximo de candidatos antes de NMS.
<code>-scheduler <modo></code>	dynamic, static-contiguous o static-round-robin.
<code>-no-annotated-video</code>	Desactiva generación del vídeo anotado.

B Dependencias

En Ubuntu/WSL, las dependencias principales son:

```
sudo apt-get update
sudo apt-get install -y \
    cmake \
    libopencv-dev \
    openmpi-bin \
    libopenmpi-dev \
    nvidia-cuda-toolkit
```